

Práctica 2

Objetivos:

Aplicar los conceptos de clase, atributos (datos miembro) y métodos (funciones miembro).

Instrucciones:

Haga uso de anotaciones, `docstring` y la convención de nombres explicada en clase de teoría para nombrar las clases, funciones, variables y métodos. Utilice notación UML para representar la clase.

- **Código:** complete la clase `Time` en un archivo `.py` o en un notebook (`.ipynb`), implementando todos los métodos y un `main()` con el menú.
- **UML:** represente la clase en notación UML usando una de estas opciones: PDF, imagen (PNG o JPG), draw.io (diagrams.net) o PlantUML.

En el esqueleto, las cabeceras de los métodos aparecen como `def nombre(...)`. Decida para cada método si es de instancia, de clase o estático, escriba sus parámetros y añada el decorador que corresponda. Justifique la decisión en un comentario junto a cada método. Tal cual está, el esqueleto no se ejecuta.

```
class Time:
    """
    Clase que representa una hora en formato AM/PM o 24 horas.

    Atributos de clase:
        TIME_FORMATS    # p. ej.: "AM", "PM", "24 HOURS"
        time_count = 0  # Cuenta el número de objetos Time creados

    Atributos:
        # Los atributos deben ser privados y exponerse mediante
        # property
        hours          # Guarda las horas
                        # (1 a 12 en AM/PM, 0 a 23 en 24 HOURS)
        minutes        # Guarda los minutos (0 a 59)
        seconds        # Guarda los segundos (0 a 59)
        time_format     # Guarda el formato: "AM", "PM" o "24 HOURS"
    """
```

```
def __init__(self):
    """
    Inicializa los atributos a 0.
    Es el único método que se le pasa parámetros como ejemplo
    """
    pass

def __assign_format(...):
    """
    Comprueba que time_format tiene un valor correcto y lo
    asigna al atributo correspondiente.
    Lo convierte a mayúsculas para evitar problemas de
    capitalización.

    Args:
        time_format: cadena con el formato de hora
                     ("AM", "PM" o "24 HOURS").

    Devuelve:
        True si el formato es correcto, False en caso
        contrario.
    """
    pass

def __is_24hour_format(...):
    """
    Comprueba si el formato de la hora es "24 HOURS".

    Devuelve:
        True si formato es "24 HOURS", False en caso contrario.
    """
    pass

def __is_valid_time(...):
    """
    Comprueba si la hora es correcta según el formato.

    Devuelve:
        True si la hora es correcta, False en caso contrario.
    """
    pass

def set_time(...):
    """
    Asigna una hora.
```

```
Args:
    hours: horas (1 a 12 en AM/PM, 0 a 23 en 24 HOURS).
    minutes: minutos (0 a 59).
    seconfs: segundos (0 a 59).
    time_format: formato ("AM", "PM" o "24 HOURS").

Devuelve:
    True si la hora se pudo asignar correctamente,
    False en caso contrario.
"""
pass

def get_time(...):
    """
    Devuelve la hora actual del objeto.
    """
    pass

def from_string(...):
    """
    Crea una nueva instancia de Time a partir de una cadena.

    Args:
        time_string: cadena con la hora en el formato
            "HH:MM:SS FORMATO", donde FORMATO es AM, PM o
            24 HOURS.

    Devuelve:
        Una instancia de Time con la hora leída.
        Si la cadena no es válida, muestra un mensaje.
    """
    # Importe el módulo 're' para expresiones regulares
    # Defina el patrón que reconoce las cadenas de hora
    # El patrón debe reconocer cadenas como
    # "14:30:00 24 HOURS" o "02:45:30 PM"
    # Use re.match para comprobar si time_string encaja

#PASOS:
# Extraiga horas, minutos, segundos y formato de los grupos

# Cree una instancia de Time

# Asigne la hora con los valores extraídos, convertidos a
# entero
```

```
# Devuélva la instancia

# Muestre mensaje si el formato de la cadena no es válido
pass

def is_valid_format(...):
    """
    Comprueba si un formato de hora dado es
    válido.

    Args:
        time_format: cadena con el formato de hora a comprobar.

    Devuelve:
        True si el formato es válido (AM, PM o 24 HOURS),
        False en caso contrario.
    """
    pass

def get_time_count(...):
    """
    Devuelve el número de objetos de la clase que se han
    creado.

    Devuelve:
        Un número entero de objetos Time creados.
    """
    pass
```

Programa principal

Cree una función fuera de la clase para mostrar la hora en una cadena con formato.

Implemente un programa controlado por menú que permita al usuario:

1. Introducir una nueva hora
2. Visualizar hora
3. Crear una hora a partir de una cadena (“HH:MM:SS FORMATO”)
4. Terminar

Use la validación de datos adecuada para garantizar que:

- El formato de hora sea válido (AM, PM o 24 HOURS)

- Los valores de hora estén dentro del rango correcto según el formato

Consideraciones adicionales:

Implemente un manejo de errores adecuado para las entradas del usuario.

- El formato de hora siempre debe almacenarse en mayúsculas, pero permita la entrada en cualquier caso.
- Para el formato de 12 horas (AM/PM), las horas deben estar entre 1 y 12.
- Para el formato de 24 HOURS, las horas deben estar entre 0 y 23.
- Los minutos y segundos siempre deben estar entre 0 y 59.

Estructura del programa

- Cree un archivo llamado *time_management.py* (o *time_management.ipynb*, si trabaja con un notebook) para la clase Time y las funciones relacionadas.
- Implemente una función *main()* que ejecute el menú. En un notebook, deje la llamada a *main()* en la última celda.